

Auditing Keep-or-Revert Decisions in Self-Improving Agents

Anonymous Authors

Abstract—When an AI coding agent keeps a revised program, later edits can build on that choice. A recipient who inherits the final program also inherits the consequences of those decisions, but its score and file hashes do not establish their evidential support. We present a decision record and offline verifier for auditing such handoffs without access to the producing agent’s runtime. The verifier connects code versions to paired test results, recalculates recorded statistics, checks declared keep-or-revert rules, and checks submitted-version identity. We characterize these checks using twelve synthetic evidence packages: nine deliberately introduced faults, two valid controls, and one internally consistent rewrite of an unsigned final reward. Format checks reject one fault; adding file-hash checks rejects three; full verification rejects all nine. Both valid controls and the reward rewrite pass. The results identify decision relationships that file binding alone cannot check. A retained real-run record illustrates missing decision evidence, but is not an additional end-to-end verifier evaluation. The contribution is a bounded audit mechanism for agent-produced artifacts: internal consistency neither proves authentic execution nor authorizes downstream reuse.

Index Terms—decision provenance, agent accountability, offline verification

I. INTRODUCTION

Suppose an AI coding agent improves a program that plays 2048. It repeatedly edits the code, tests it, and keeps the new version or returns to an earlier one. RSI-Exam provides this setting: agents improve task programs using visible evaluations, then submit them for grading on unseen data [1]. Here, self-improving means improving the task program, not the language model’s weights.

Keeping a candidate makes it the starting point for later edits. A receiving agent or researcher who continues from the final program inherits the consequences of those choices, often without access to the producing runtime. The recipient needs to ask: Which parent was each candidate compared against? Which tests belonged to each version? Did the declared rule support keeping it? Was a required confirmation completed? Is the submitted program the version that was selected?

Logs may contain answers, but their entries need not agree. File hashes associate records with supplied bytes; they can match even when a recorded score difference is arithmetically wrong. We study *decision consistency*: whether supplied measurements and the declared selection rule support the recorded keep-or-revert decision. This does not recover private reasoning or prove that measurements genuinely came from an execution.

Our question is: *what can a recipient check about a recorded version decision beyond file format and hash agreement?* The

audit unit is a version-selection decision and its evidence, not an inventory of tool calls. We contribute an executable decision record and offline checker, characterized through controlled changes to evidence packages. Handoff motivates the recipient’s role; we test packages, not a multi-agent network. This is a task-specific systems contribution, not a general theory of provenance or a score-improvement method.

A. A real record shows the evidence gap

A retained record from an actual RSI run makes the problem concrete. A candidate reports a higher visible mean score than its parent, 48115 versus 38974.5, but is marked reverted without a recorded evaluation under the selection rule. The submitted version instead branches from that parent. The higher reported score alone cannot explain this choice, nor does it prove that reverting was wrong.

We can verify the retained experiment log’s hash and recover the declared branch history. But original code snapshots and per-seed measurements are absent from this retained export, so we cannot check their bytes or recalculate the decision’s statistical support. The companion’s Case R inspection reports these limits explicitly. This real-run example motivates what must be captured; the controlled experiment below tests the verifier on a different, fully supplied synthetic package.

II. RECORDING AND CHECKING A DECISION

A. What the producer hands over

The evidence package contains saved code versions, test-result files, a decision log, submitted code, and a machine-readable summary called a *capsule*. The producer records decisions during the run and builds the capsule afterward. The recipient checks this directory without accessing the agent’s runtime or rerunning its program.

Each decision identifies five things:

- 1) The *parent* version being compared against and the new *candidate*.
- 2) The parent and candidate result files, labeled by role and bound by file hashes.
- 3) The score direction (higher or lower is better), statistic, confidence-interval procedure, random seed, and threshold.
- 4) The recorded calculation and decision: keep, revert, or await a confirmation evaluation.
- 5) A later confirmation decision, if one is required to resolve that pending state.

The capsule also links the submitted program to a recorded version. A version occurrence is distinct from its code contents: reverting can create another occurrence of the same code. Code identity follows the task’s staging rules, excluding Python bytecode caches; a separate whole-directory hash checks the supplied tree. These conventions avoid confusing a cache change with a program change.

B. A worked decision, from scores to submission

Table I follows the synthetic package used in our controlled experiment. v1 is the parent. Candidate v2 performs worse and is reverted. Candidate v3 is also derived from v1: its initial result is inconclusive, but a subsequent confirmation supports keeping it. The submitted program is v3.

TABLE I

RUNNING EXAMPLE FROM THE SYNTHETIC PACKAGE, NOT A REAL-RUN RESULT. EACH ROW COMPARES THE CANDIDATE WITH v1.

Candidate/test	Mean delta	90% interval	Decision
v2 / initial	-318.75	[-382.5, -258.75]	Revert
v3 / initial	260	[-30, 583.75]	Pending
v3 / confirm	518.75	[467.5, 575]	Keep

The calculations use paired games: parent and candidate are evaluated on the same seed within each comparison. For a higher-is-better score, the mean improvement is

$$\hat{\Delta} = \frac{1}{n} \sum_{i=1}^n (s_{\text{candidate}}(i) - s_{\text{parent}}(i)).$$

The interval expresses uncertainty around the estimated improvement. The verifier recomputes the estimate and percentile-bootstrap interval from the supplied per-seed results and declared algorithm parameters. The example uses eight paired scores, 5000 resamples, and bootstrap seed 20260902. For a lower-is-better score the differences change sign.

In this package’s declared policy, an interval entirely below zero leads to revert, one entirely above zero supports keep, and an interval spanning zero requires confirmation. Thus the initial v3 result cannot by itself close the decision. Removing its confirmation while still submitting v3 leaves an unresolved pending decision. More generally, the verifier checks the recorded policy, not whether that policy is scientifically optimal.

C. What the recipient checks

The checks answer progressively stronger questions:

- 1) **Format:** Are required fields present and values well formed?
- 2) **File binding:** Do referenced files exist, and do their bytes match their declared hashes?
- 3) **Decision consistency:** Do parent/candidate references, recalculated statistics, decision rules, and submitted-version identity agree?

Full verification also reports *coverage*: whether every snapshot in the supplied version directory appears in the record. An

unrecorded extra snapshot can leave the other checks passing but make coverage partial. Our experiment requires complete coverage for acceptance. This is completeness of the supplied directory, not proof that the producer disclosed every experiment.

Consider changing v2’s recorded mean delta from -318.75 to -317.75, in both the decision log and capsule, then updating the log hash. Format and file binding still pass. Recalculation detects the discrepancy. This particular one-point error would not change the revert verdict; it tests numerical consistency, not the discovery of an incorrectly chosen winner. A separate test changes the submitted code while retaining the old version claim, exercising decision-to-artifact identity.

III. CONTROLLED EVALUATION

A. What is being tested?

We test the checking procedure, not the agent’s ability to play 2048. No new agent rollout is used for this experiment. We copy the same committed synthetic package twelve times and apply one specified change per case (or none for the clean control). Each copy must pass full verification before modification.

The twelve cases comprise nine deliberately introduced faults, two valid controls, and one authenticity-boundary control. Their manifest was committed before first execution, but after inspecting the implementation and development tests. This is an author-designed characterization, not an independent or held-out benchmark. The implementation is unchanged by the study.

We run three checking configurations on each package: **S** checks format only; **B** adds file bindings; **V** performs full verification and requires complete coverage. B uses the production hash functions but omits recalculation, decision rules, cross-field identity and coverage. These are internal ablations of our checker, not measurements of competing systems.

To avoid testing only stale hashes, relevant decision changes are mirrored in the log and capsule and the log hash is refreshed. Swapping the parent/candidate measurements exchanges the referenced paths and hashes while retaining their role labels. Changing the submitted program also refreshes its hashes while retaining the claimed version. All cases, changed-file hashes and actual diagnostics are retained in the companion.

B. Results and interpretation

Table II reports every outcome. S rejects one of nine faults, B rejects three, and V rejects nine. The six additional refusals cover three measurement inconsistencies, an unresolved confirmation, a submitted-version mismatch, and an incomplete snapshot inventory. Thus files can match their declarations while the relationships needed to audit a decision still fail.

Both valid controls pass: the unchanged package and a package whose excluded bytecode caches differ. This establishes acceptance on these two examples, not a population false-positive rate. Likewise, the fault counts do not estimate detection rates on naturally occurring errors.

TABLE II

CONTROLLED PACKAGES. S: FORMAT; B: FORMAT + FILE HASHES; V: FULL CHECKS INCLUDING COVERAGE. A: ACCEPTS THAT CHECK SCOPE; R: REJECTS.

Package change	S	B	V
None (valid control)	A	A	A
Excluded caches differ; full hashes refreshed	A	A	A
Reward and declaration rewritten together	A	A	A
Unrecognized version status	R	R	R
Referenced measurement file removed	A	R	R
Recorded score direction reversed	A	A	R
Recorded mean delta increased by one	A	A	R
Parent/candidate measurements exchanged	A	A	R
Pending submission left unconfirmed	A	A	R
Submission changed; version claim retained	A	A	R
Recorded snapshot removed	A	R	R
Unrecorded snapshot added	A	A	R

Rejection reasons matter. The extra snapshot is rejected for partial coverage, not an integrity error. The missing snapshot is correctly refused, but its diagnostic prefix differs from the manifest’s expected prefix. All nine faults are refused, but only eight match the prespecified diagnostic target. The unchanged expectation and actual output are both preserved; unexpected runtime exceptions abort the study rather than count as detections.

C. A passing rewrite defines the trust boundary

The final control changes an unsigned final reward file and the capsule’s matching reward to 0.5, then refreshes the file hash. V accepts. This is the final benchmark reward, not one of the paired visible-game measurements used for keep-or-revert decisions. The verifier checks agreement with the supplied reward file; it neither authenticates the grader nor recalculates that reward from underlying score details.

This accepted rewrite is important to the interpretation of the nine refusals. Our mechanism checks consistency among supplied declarations and bytes. A producer able to rewrite evidence consistently can still supply false material. Acceptance must therefore not be read as proof that these experiments really happened or that the agent is trustworthy.

IV. SCOPE, RELATED WORK, AND LIMITATIONS

Capture, checking, and admission. Capture retains recorded versions, measurements and decisions. Checking tests their consistency. Admission determines whether a recipient is authorized to rely on a result in its own context. We implement and characterize checking, specifying the evidence it needs, but establish no admission policy. A reviewer can inspect a particular decision and its named failures rather than treating an entire log as a single trust claim.

PROV-CONSTRAINTS already defines consistency conditions over provenance histories [2]. Our narrower contribution is a working profile for agent version selection: paired-result recalculation, confirmation state, and the link from a decision to submitted code. We do not claim that adding a provenance record is itself novel.

in-toto verifies cryptographically linked software-supply-chain steps [3]; our unsigned packages provide no comparable authentication. MLflow records experiment parameters, metrics and artifacts [4]; a tracking system could retain the inputs our checks need. Neither system is an experimental baseline here. Storage, authentication, and decision-consistency checks address different parts of an evidence handoff.

Our results use one synthetic base, author-selected changes and shared checking components. The retained real-run case shows why evidence capture matters, but missing source files prevent its full replay. We have not demonstrated generalization to unseen tasks, reduced human audit effort, independent implementer agreement, or resistance to coordinated falsification. Reproducing an interval also does not certify its statistical validity under adaptive reuse of test seeds. A decision can follow its declared rule and still perform poorly on unseen games. Running checks outside the producing runtime does not authenticate the files that runtime supplies.

The Agent-Native Research Artifact companion [5] contains cases, claims, source bindings and exploration history. Historical reward comparisons, development diagnostics and a legacy reuse case remain there, not as additional verifier trials. In a networked workflow, check outcomes could inform a recipient’s decision to continue from a supplied version. Identity, authorization and network-scale governance remain outside this evaluation.

V. CONCLUSION

Later work can build on an agent’s keep decisions. Our record and verifier let a recipient check whether the supplied measurements, decision rule and submitted code agree. Full checks reject six authored faults that format and file-hash checks accept; a consistent unsigned reward rewrite still passes. These checks make decision relationships auditable within a supplied package. Authentic execution, better future performance and permission to reuse remain separate questions.

REFERENCES

- [1] Aiming Lab, *RSI-Exam*, 2026. [Online]. Available: <https://github.com/aiming-lab/RSI-Exam>
- [2] J. Cheney, P. Missier, and L. Moreau, Eds., *Constraints of the PROV Data Model*, W3C Recommendation, Apr. 2013.
- [3] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappel, *in-toto: Providing farm-to-table guarantees for bits and bytes*, in *USENIX Security*, 2019, pp. 1393–1410.
- [4] MLflow, *ML Experiment Tracking*. [Online]. Available: <https://mlflow.org/docs/latest/ml/tracking/>
- [5] ARA Labs, *Agent-Native Research Artifact*. [Online]. Available: <https://github.com/ARA-Labs/Agent-Native-Research-Artifact>